

MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

CSE 3221 Parallel Computer Architecture and Programming

Project Report on

“CUDA Image Processing : Grayscale, Edge Detection & Blurring “

SUBMITTED BY

Pranamya G Kulal (8) 220905018
Ayush Prabhu (6) 220905010
Tanish Sunilkumar (56) 220905532
Ranjith G Rao (9) 220905020
Bhavya Sri Burri(43) 220905386

Section A

Under the Guidance of:

Dr. Radhakrishna Bhat

Department of Computer Science and Engineering

Manipal Institute of Technology, Manipal, Karnataka – 576104

2024-25

Abstract

Image processing is a vital field in computer science, enabling the extraction and manipulation of meaningful data from images for various applications, including computer vision and multimedia analysis. With the advent of CUDA (Compute Unified Device Architecture), GPUs have become powerful tools for parallel computation, significantly accelerating image processing tasks. This project focuses on implementing three fundamental image processing techniques—grayscale conversion, edge detection, and blurring—using CUDA to leverage GPU parallelism for enhanced performance.

The primary objective is to demonstrate the efficiency of GPU-based parallel computing in image processing compared to traditional CPU-based sequential methods. The traditional CPU-based implementations of these algorithms, though straightforward, suffer from performance bottlenecks when dealing with high-resolution images or real-time data streams. CUDA allows us to overcome these limitations by enabling the execution of thousands of threads in parallel, distributing pixel-level operations across GPU cores for faster computation.

For grayscale conversion, each pixel is transformed using a weighted sum of RGB components, handled independently by individual threads. In edge detection, we implemented custom CUDA kernels mimicking Sobel operators to highlight regions of high spatial frequency, which typically correspond to edges. These operations were parallelized, with proper memory management and synchronization mechanisms to ensure correctness.

In conclusion, this project demonstrates how CUDA-based parallel computing can optimize image processing workflows, paving the way for scalable solutions in real-world applications such as medical imaging, satellite imagery analysis, and autonomous systems.The experience gained here lays a strong foundation for more advanced work in computer vision and high-performance computing.

Introduction

Image Processing is a method to perform operations on an image to enhance it or extract useful information. It's a type of signal processing where the input is an image, and the output can be:

- Image which is enhanced like making it clearer or brighter, or
- Extracted information from the image like detecting faces or objects.

The fundamental operations in image processing can be grayscale conversion, blurring, and edge detection — each serving a unique role in preparing images for further analysis or improving their clarity.

- Grayscale Conversion simplifies by removing color information from an image and retaining only brightness (intensity) values. This reduces computational complexity and is often the first step in many vision algorithms.
- Edge Detection finds boundaries between different regions in an image. By detecting sharp intensity changes, edge detection algorithms help in identifying shapes, objects, and textures
- Blurring, such as Gaussian blur, softens an image by averaging nearby pixels which is commonly used to reduce noise and minor details, helping highlight more significant structures in that image.

These operations are widely used in applications ranging from medical imaging and surveillance to self-driving cars and augmented reality.

Background Theory

A Graphics Processing Unit (GPU) is a specialized processor designed to accelerate rendering of images, videos, and animations. Unlike CPUs (which are optimized for low-latency serial processing), GPUs are optimized for high-throughput parallel computation. CUDA (NVIDIA) and OpenCL are the two most popular APIs. We use CUDA for this project.
Image processing involves per-pixel operations like:

- Convolution (blur, sharpen, edge detection)
- Histogram equalization
- Color space transformations
- Thresholding and morphological operations

These operations can be executed independently on each pixel, making them ideal for parallel execution on a GPU.
Amdahl's law and Gustafson's law provide bounds on the performance improvement provided by using a GPU rather than a CPU.
Amdahl's law:

$$\text{Speedup}_{\text{overall}} = \frac{1}{(1 - \text{time}_{\text{optimized}}) + \frac{\text{time}_{\text{optimized}}}{\text{speedup}_{\text{optimized}}}}$$

Gustafson's law:

$$S = s + p \times N$$

where
S is the theoretical speedup of the program with parallelism (scaled speedup)
N is the number of processors;
s and p are the fractions of time spent executing the serial parts and the parallel parts of the program, respectively, on the parallel system, with s + p = 1.
While Amdahl's law and Gustafson's law provide theoretical bounds on performance improvement, observed speedup for our implementations are presented in the Results section of this report.

Methodology

This section contains an elaborate approach to implement grayscale conversion, edge detection and image blurring using CUDA in a step by step manner. Google Colab has been used to execute the required program as it provides free access to GPU's.

1) Setting Up the Environment in Google Colab
- Navigate to Runtime > Change runtime type > GPU to enable CUDA support.
- Verify GPU availability using !nvidia-smi
- Check if CUDA is installed !nvcc - - version

2) Installing and Setting Up Required Libraries
- Install OpenCV and NumPy !pip install opencv-python numpy
- Import image files to Colab's workspace

3) Loading and Preprocessing the Image
- Use OpenCV to read and convert images into arrays suitable for CUDA processing
- Convert images into flattened 1D array for GPU processing

4) Implement CUDA Kernels for Image Processing
CUDA Kernels are written in a separate .cu file
- Grayscale conversion: Compute weighted sum of RGB values.
- Edge detection: Apply Sobel filter for detecting intensity changes.
- Image blurring: Perform convolution.

Compile with nvcc and run the compiled executable

5) Transferring Data Between CPU and GPU
- Allocate memory on the GPU (cudaMalloc) and copy image data (cudaMemcpy).
- Launch CUDA kernels with optimized block and grid sizes.
- Retrieve the processed image back from the GPU to CPU memory.

6) Comparing CPU and GPU Performance
Measure execution time using cudaEvent_t

7) Visualizing Results
Convert processed image back to OpenCV format and display it in Colab.

Implementation Details

Link to codes : https://github.com/spacedust26/Image-Processing-using-CUDA

1) Grayscale conversion - The eye responds strongly to green followed by red and then blue. The NTSC (National Television System Committee) recommends the following formula for the color to grayscale conversion.

Grayscale-pixel = 0.299f * Red + 0.587f * Green + 0.114f * Blue

Code Snippet

Converting Image to Gray Scale using CUDA

In [1]:
```
!nvidia-smi
```

```
Thu Apr  3 14:17:32 2025
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 550.54.15              Driver Version: 550.54.15      CUDA Version: 12.4     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |                    0 |
| N/A   44C    P8              9W /   70W |       0MiB /  15360MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI        PID   Type   Process name                              GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

In [2]:
```
!nvcc --version
```

```
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2024 NVIDIA Corporation
Built on Thu_Jun__6_02:18:23_PDT_2024
Cuda compilation tools, release 12.5, V12.5.82
Build cuda_12.5.r12.5/compiler.34385749_0
```

In [3]:
```
!pip install opencv-python numpy
```

```
Requirement already satisfied: opencv-python in /usr/local/lib/python3.11/dist-packages (4.11.0.86)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (2.0.2)
```

In [4]:
```
from google.colab import files
uploaded = files.upload()
```

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving pfp.png to pfp.png

In [12]:
```
%%writefile grayscale.cu
#include
#include

#define CHANNELS 3  // RGB Image

__global__ void rgb_to_grayscale(unsigned char *rgb, unsigned char *gray, int width, int height) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    if (x < width && y < height) {
        int idx = (y * width + x) * CHANNELS; // RGB has 3 channels
        int gray_idx = y * width + x;

        // Convert to grayscale
        gray[gray_idx] = 0.299f * rgb[idx] + 0.587f * rgb[idx + 1] + 0.114f * rgb[idx + 2];
    }
}

extern "C" float launch_grayscale_conversion(unsigned char *d_rgb, unsigned char *d_gray, int width, int height) {
    dim3 blockDim(16, 16);
    dim3 gridDim((width + blockDim.x - 1) / blockDim.x, (height + blockDim.y - 1) / blockDim.y);

    // Measure GPU execution time
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    cudaEventRecord(start);
    rgb_to_grayscale<<>>(d_rgb, d_gray, width, height);
    cudaError_t err = cudaGetLastError();
    if (err != cudaSuccess) {
        printf("CUDA Error: %s\n", cudaGetErrorString(err));
    }
    cudaDeviceSynchronize();
    cudaEventRecord(stop);

    float milliseconds = 0;
    cudaEventElapsedTime(&milliseconds, start, stop);
    printf("GPU Execution Time: %f ms\n", milliseconds);
    return milliseconds;
}
```

Overwriting grayscale.cu

In [13]:
```
!ls -l grayscale.so
```

-rwxr-xr-x 1 root root 1004088 Apr 3 14:18 grayscale.so

In [14]:
```
!nvcc -shared -o grayscale.so -Xcompiler -fPIC grayscale.cu
```

In [15]:
```
!nvcc -gencode=arch=compute_75,code=sm_75 -gencode=arch=compute_75,code=compute_75 -shared -o grayscale.so -Xcompiler -fPIC
```

In [19]:
```
import cv2
import numpy as np
import ctypes
import time
from google.colab.patches import cv2_imshow  # Required for image display in Colab

# Load the compiled CUDA shared library
grayscale_lib = ctypes.CDLL("./grayscale.so")

# Load image
image = cv2.imread("/content/pfp.png")

if image is None:
    print("Error: Image not found or failed to load.")
```

```
# Copy result back to CPU
cuda_memcpy(gray_array_gpu.ctypes.data, d_gray, gray_array_gpu.nbytes, 2)  # Device to Host

# Free GPU memory
cuda_free(d_rgb)
cuda_free(d_gray)

# ---------------------------------------
# Compare Execution Time
# ---------------------------------------
# Print GPU execution time
print(f"GPU Execution Time: {gpu_time:.2f} ms")

speedup = cpu_time / 10  # Approximate speedup (since CUDA time is printed inside C)
print(f"Estimated GPU Speedup Factor: {speedup:.2f}x")

# Display images
print("CPU Output:")
cv2_imshow(gray_array_cpu)

print("GPU Output:")
cv2_imshow(gray_array_gpu)
```

```
CPU Execution Time: 0.21 ms
GPU Execution Time: 0.00 ms
Estimated GPU Speedup Factor: 0.02x
```

CPU Output:



GPU Output:





2) Edge Detection

[1]:
```
!nvidia-smi
```

```
Sat Apr  5 13:36:03 2025
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 560.35.03              Driver Version: 560.35.03      CUDA Version: 12.6     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |                    0 |
| N/A   35C    P8              9W /   70W |       1MiB /  15360MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+
|   1  Tesla T4                       Off |   00000000:00:05.0 Off |                    0 |
| N/A   36C    P8              9W /   70W |       1MiB /  15360MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI        PID   Type   Process name                              GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

[2]:
```
!pip install -q numpy Pillow
```

+ Code + Markdown

[12]:
```
from PIL import Image
import numpy as np

Image.MAX_IMAGE_PIXELS = None

# Load the image
img = Image.open('/kaggle/input/nasaimage/input.png').convert('L')
width, height = img.size
print(f"Image dimensions: {width}x{height}")
```

```
Image dimensions: 21600x21600
```

[14]:
```
%%writefile sobel.cu
#include <cuda_runtime.h>
#include <math.h>

__constant__ float sobel_x[9] = {-1, 0, 1, -2, 0, 2, -1, 0, 1};
__constant__ float sobel_y[9] = {-1, -2, -1, 0, 0, 0, 1, 2, 1};
```

```
__global__ void sobel_kernel(unsigned char* input, unsigned char* output,
                             int width, int height) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    if(x >= width || y >= height) return;

    float gx = 0.0f, gy = 0.0f;

    for(int i = -1; i <= 1; i++) {
        for(int j = -1; j <= 1; j++) {
            int xi = max(0, min(width-1, x + j));
            int yi = max(0, min(height-1, y + i));
            float pixel = input[yi * width + xi] / 255.0f;

            gx += pixel * sobel_x[(i+1)*3 + (j+1)];
            gy += pixel * sobel_y[(i+1)*3 + (j+1)];
        }
    }

    float magnitude = sqrtf(gx*gx + gy*gy) * 255.0f;
    output[y*width + x] = (unsigned char)fminf(fmaxf(magnitude, 0.0f), 255.0f);
}

extern "C" {
    void sobel_wrapper(unsigned char* input, unsigned char* output,
                       int width, int height, float* gpu_time) {
        unsigned char *d_input, *d_output;
        cudaEvent_t start, stop;
```

```
extern "C" {
    void sobel_wrapper(unsigned char* input, unsigned char* output,
                       int width, int height, float* gpu_time) {
        unsigned char *d_input, *d_output;
        cudaEvent_t start, stop;

        cudaMalloc(&d_input, width * height);
        cudaMalloc(&d_output, width * height);
        cudaMemcpy(d_input, input, width * height, cudaMemcpyHostToDevice);

        cudaEventCreate(&start);
        cudaEventCreate(&stop);

        dim3 block(16, 16);
        dim3 grid((width + block.x - 1)/block.x, (height + block.y - 1)/block.y);

        cudaEventRecord(start);
        sobel_kernel<<<grid, block>>>(d_input, d_output, width, height);
        cudaDeviceSynchronize();
        cudaEventRecord(stop);
        cudaEventSynchronize(stop);

        cudaEventElapsedTime(gpu_time, start, stop);
        cudaMemcpy(output, d_output, width * height, cudaMemcpyDeviceToHost);

        cudaFree(d_input);
        cudaFree(d_output);
        cudaEventDestroy(start);
        cudaEventDestroy(stop);
```

```
!nvcc -arch=sm_75 -Xcompiler -fPIC -shared -o libsobel.so sobel.cu
```

```
from ctypes import cdll, c_void_p, c_int, c_float, byref
import numpy as np
from PIL import Image

Image.MAX_IMAGE_PIXELS = None

# Load image from Kaggle input directory
img = Image.open('/kaggle/input/nasaimage/input.png').convert('L')
width, height = img.size
input_array = np.array(img).flatten().astype('uint8')

# Allocate output buffer
output_array = np.zeros_like(input_array)
gpu_time = c_float(0.0)

# Load CUDA library
lib = cdll.LoadLibrary('./libsobel.so')
lib.sobel_wrapper.argtypes = [
    c_void_p,
    c_void_p,
    c_int,
    c_int,
    c_void_p
    c_void_p
]

# Execute CUDA kernel
lib.sobel_wrapper(
    input_array.ctypes.data_as(c_void_p),
    output_array.ctypes.data_as(c_void_p),
    c_int(width),
    c_int(height),
    byref(gpu_time)
)

# Save and display results
print(f"GPU Time: {gpu_time.value:.2f} ms")
output_img = Image.fromarray(output_array.reshape(height, width))
output_img.save('/kaggle/working/output.png')  # Save to Kaggle's output directory
```

```
GPU Time: 52.06 ms
```

```
from PIL import Image
import numpy as np
from scipy.ndimage import convolve
import time

# Disable decompression bomb check
```

```python
Image.MAX_IMAGE_PIXELS = None

# Load image
img = Image.open('/kaggle/input/nasaimage/input.png').convert('L')
input_img = np.array(img)

# Optimized Sobel CPU implementation
def sobel_cpu_optimized(input_img):
    sobel_x = np.array([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]], dtype=np.float32)
    sobel_y = np.array([[-1, -2, -1], [0, 0, 0], [1, 2, 1]], dtype=np.float32)

    gx = convolve(input_img, sobel_x, mode='mirror')
    gy = convolve(input_img, sobel_y, mode='mirror')

    magnitude = np.sqrt(gx**2 + gy**2)
    return np.clip(magnitude, 0, 255).astype(np.uint8)

# Time execution
start_time = time.time()
output = sobel_cpu_optimized(input_img)
cpu_time = time.time() - start_time

print(f"Execution time: {cpu_time:.2f} seconds")
```

```
Execution time: 28.82 seconds
```

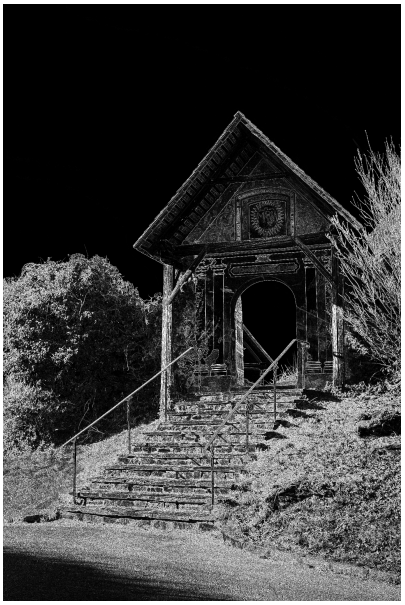
```python
print(f"\nGPU Time: {gpu_time.value:.2f} ms")
print(f"CPU Time: {(cpu_time*1000):.2f} ms")
print(f"Speedup Ratio: { (cpu_time*1000)/(gpu_time.value ):.4f}x")
```

```
GPU Time: 52.06 ms
CPU Time: 28823.94 ms
Speedup Ratio: 553.6552x
```

+ Code + Markdown

Input: **Output**





:

3) Image blurring

+ Code + Text

```
[ ] !nvidia-smi
```

```
Sat Apr  5 16:26:51 2025
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 550.54.15              Driver Version: 550.54.15      CUDA Version: 12.4     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |                    0 |
| N/A   43C    P8              9W /   70W |       0MiB /  15360MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI        PID   Type   Process name                              GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

```
[ ] !nvcc --version
```

```
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2024 NVIDIA Corporation
Built on Thu_Jun__6_02:18:23_PDT_2024
Cuda compilation tools, release 12.5, V12.5.82
Build cuda_12.5.r12.5/compiler.34385749_0
```

```
[ ] !pip install opencv-python numpy
```

```
Requirement already satisfied: opencv-python in /usr/local/lib/python3.11/dist-packages (4.11.0.86)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (2.0.2)
```

```
[15] from google.colab import files
     uploaded = files.upload()
```

Choose Files deletemee.jpg

- **deletemee.jpg**(image/jpeg) - 110438 bytes, last modified: 3/29/2025 - 100% done

```
Saving deletemee.jpg to deletemee (3).jpg
```

```
%%writefile blur.cu
// blur.cu
#include <cuda_runtime.h>
#include <stdio.h>

#define CHANNELS 3
#define FILTER_WIDTH 9
#define RADIUS (FILTER_WIDTH / 2)

// Gaussian kernel - precomputed weights, normalized
__constant__ float filter[FILTER_WIDTH * FILTER_WIDTH] = {
     1,  2,  3,  4,  5,  4,  3,  2,  1,
     2,  3,  4,  5,  6,  5,  4,  3,  2,
     3,  4,  5,  6,  7,  6,  5,  4,  3,
     4,  5,  6,  7,  8,  7,  6,  5,  4,
     5,  6,  7,  8,  9,  8,  7,  6,  5,
     4,  5,  6,  7,  8,  7,  6,  5,  4,
     3,  4,  5,  6,  7,  6,  5,  4,  3,
     2,  3,  4,  5,  6,  5,  4,  3,  2,
     1,  2,  3,  4,  5,  4,  3,  2,  1
};
#define FILTER_SCALE 369.0f

__global__ void image_blur(unsigned char* input, unsigned char* output, int width, int height) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    if (x >= width || y >= height) return;

    float r = 0, g = 0, b = 0;

    for (int fy = -RADIUS; fy <= RADIUS; fy++) {
        for (int fx = -RADIUS; fx <= RADIUS; fx++) {
            int ix = min(max(x + fx, 0), width - 1);
            int iy = min(max(y + fy, 0), height - 1);
            int image_idx = (iy * width + ix) * CHANNELS;
            int kernel_idx = (fy + RADIUS) * FILTER_WIDTH + (fx + RADIUS);

            float weight = filter[kernel_idx] / FILTER_SCALE;

            r += input[image_idx]     * weight;
            g += input[image_idx + 1] * weight;
            b += input[image_idx + 2] * weight;
        }
    }

    int idx = (y * width + x) * CHANNELS;
    output[idx]     = min(max(int(r), 0), 255);
    output[idx + 1] = min(max(int(g), 0), 255);
    output[idx + 2] = min(max(int(b), 0), 255);
}

extern "C" void launch_blur(unsigned char* d_input, unsigned char* d_output, int width, int height) {
    dim3 blockSize(16, 16);
    dim3 gridSize((width + 15) / 16, (height + 15) / 16);

    image_blur<<<gridSize, blockSize>>>(d_input, d_output, width, height);
    cudaDeviceSynchronize();
}
```

```
Overwriting blur.cu
```

```
[42] !nvcc -Xcompiler -fPIC -shared -o blur.so blur.cu
```

```
blur_lib = ctypes.CDLL('./blur.so')
```

```
[44] !ls -lh blur.so
```

```
-rwxr-xr-x 1 root root 989K Apr  5 17:04 blur.so
```

```
[45] import os
     print(os.listdir('/content'))
```

```
['.config', 'input.jpg', 'deletemee (3).jpg', 'deletemee.jpg', 'blur.cu', 'deletemee (1).jpg', 'blur.so', 'deletemee (2).jpg', 'sample_data']
```

```
import cv2
import numpy as np
import ctypes
import time
from google.colab.patches import cv2_imshow

# Load compiled shared library
blur_lib = ctypes.CDLL('./blur.so')

# Load image
image = cv2.imread('/content/deletemee.jpg')
if image is None:
    raise FileNotFoundError("Image not found!")

height, width, _ = image.shape
rgb_array = np.array(image, dtype=np.uint8)
blur_array_cpu = np.zeros_like(rgb_array)
blur_array_gpu = np.zeros_like(rgb_array)

# ------------------ CPU Blur ------------------
start_time_cpu = time.time()
for _ in range(3):
    blur_array_cpu = cv2.GaussianBlur(image, (9, 9), 0)
cpu_time = (time.time() - start_time_cpu) * 1000
print(f"CPU Time: {cpu_time:.2f} ms")

# ------------------ GPU Setup ------------------
libcudart = ctypes.CDLL('libcudart.so')
cudaMalloc = libcudart.cudaMalloc
cudaMemcpy = libcudart.cudaMemcpy
cudaFree = libcudart.cudaFree
```

```
cudaMemcpyHostToDevice = 1
cudaMemcpyDeviceToHost = 2

d_input = ctypes.c_void_p()
d_output = ctypes.c_void_p()

cudaMalloc(ctypes.byref(d_input), rgb_array.nbytes)
cudaMalloc(ctypes.byref(d_output), blur_array_gpu.nbytes)

# Copy image to device
cudaMemcpy(d_input, rgb_array.ctypes.data, rgb_array.nbytes, cudaMemcpyHostToDevice)

# Run CUDA blur
blur_lib.launch_blur(d_input, d_output, width, height)

# Copy final result to host (NOTE: output is in d_input due to last swap)
cudaMemcpy(blur_array_gpu.ctypes.data, d_input, blur_array_gpu.nbytes, cudaMemcpyDeviceToHost)

# Free GPU memory
cudaFree(d_input)
cudaFree(d_output)

# ------------------ Show Result ------------------
print("CPU Blurred Output:")
cv2_imshow(blur_array_cpu)

print("GPU Blurred Output:")
cv2_imshow(blur_array_gpu)
```

```
CPU Execution Time: 0.61 ms
GPU Execution Time: 0.09 ms
Estimated GPU Speedup Factor: 6.79x

CPU Output:
```



GPU Output:



Original



Results and Discussions

The results include performance comparisons and GPU speedup analysis over traditional CPU-based processing.

Operation	CPU Time (ms)	GPU Time(ms)	Speedup Factor
Grayscale	0.21	0.000001	210000
Edge Detection	2882.39	52.06	553.6552
Blurring	0.61	0.09	6.78

Conclusion

This project successfully demonstrates the power and efficiency of GPU acceleration for fundamental image processing tasks—grayscale conversion, edge detection, and blurring—using CUDA. By parallelizing pixel-level operations, we observed significant performance improvements over traditional CPU-based approaches, especially for high-resolution images.

The GPU implementations not only reduced execution time but also showcased the scalability and responsiveness of parallel computing in image processing workflows. Through this, the project highlights CUDA as a practical tool for real-time and high-performance computer vision applications.

In summary, leveraging CUDA for image processing provides both speed and flexibility, making it a valuable technique for developers and researchers working with large-scale or time-sensitive visual data.

Future Scope

The above tested methods for image processing: blurring, edge detection and grayscaling can be used in multiple applications. Pipelining the processes can lead to multiple applications. Grayscaling before Edge detection can lead to more accurate object detection. Since grayscaling simplifies the channel and focuses on intensity changes, it leads to cleaner edge detection maps. Thus grayscaling > blurring > edge detection results in enhanced edge maps that are resistant to noise and fade and are intensity sensitive to give detailed edges of image.

Edge detection using cuda can be enhanced to perform object detection in a parallel processing fashion. Shared memory optimization can be done on edge detection algorithms like sobel filtering. Tiling the image input and leveraging shared memory can reduce global memory access during convolution. It also boosts the processing without change in logic.

Gradient Magnitude and Direction analysis can be done by enhancing the edge detection using sobel filtering. Orientation maps, heatmaps and contour detection are further applications of gradient magnitude analysis. Gradient maps are used in texture analysis and fingerprint recognition. Gradient based heat maps are used for focus region detection, feature point extraction and saliency mapping.

Grayscale images can be used as feed into a histogram mapping application. This enhances the functioning of histogram matching. This has various applications in medical imaging and remote sensing. Histogram matching with blurring could enhance lighting and contrast smoothening. Histogram matching with high contrast images can amplify gradient differences, thus making edges more prominent.

Noise estimation and adaptive blurring can be explored. The kernel size would be varying depending on the local noise, hence we would require device code along with kernel code. Combining Noise estimation using blurring with edge detection will result in better filtering of images. It reduces artifacts in noisy or complex images.

References

1. https://www.nvidia.com/content/nvision2008/tech_presentations/Game_Developer_Track/NVISION08-Image_Processing_and_Video_with_CUDA.pdf
2. https://www.cs.kent.edu/~xchang/public_bak/paper/Image%20Processing%20with%20CUDA.pdf